

Practice Question Bank

Structural Design Patterns

Total Questions: 8 — Recommended Time: 4.5 Hours

Question 1: Hardware PC Builder

Duration: 35 minutes

A computer hardware shop sells individual components such as Graphics Cards (e.g., RTX 5060, RTX 4070 Super), RAM (e.g., 16GB DDR5 6000MHz), and Motherboards. Recently, the shop started offering “Custom PC Builds.” A Custom Build is essentially a package that contains multiple individual components. Customers can also nest smaller packages (e.g., a “Cooling Kit” containing thermal paste and fans) inside their main Custom PC Build.

By default, everything is packed in standard cardboard boxes. However, customers can upgrade their packaging at checkout:

- **Secure Transit Packaging:** Adds 500 BDT. Uses extra bubble wrap and foam inserts.
- **Premium RGB Showcase Box:** Adds 1500 BDT. The PC is delivered in a transparent acrylic case with LED lighting.

Additionally, the shop offers an **Express Assembly Service** which adds a flat 2000 BDT to the total cost of any custom build or component bundle.

Your Task:

Design and implement this system. A customer order could include individual components, nested kits, and full custom builds. During checkout, the system must process all items uniformly to calculate the final price and display a hierarchical receipt of all components. Support independent extensions of the packaging and assembly services.

Question 2: Legacy E-Commerce API

Duration: 30 minutes

You are working on the backend of a grocery delivery application. The inventory management subsystem currently uses a legacy 3rd-party library that fetches stock details exclusively in a proprietary `InventoryData` format (a custom flat-file string format). However, your newly developed frontend client requires the data strictly in JSON format. Additionally, based on the network bandwidth and security context of the client's request, the client can optionally demand that the JSON response be:

- **Compressed** (using GZIP to reduce payload size).
- **Encrypted** (using AES-256 for secure transmission).

The client can request Compression, Encryption, both, or neither dynamically at runtime.

Your Task:

Implement the system using structural patterns. You cannot alter the existing legacy library that returns the flat-file string. Your design must dynamically apply the requested transformations (to JSON, then optionally compress/encrypt) in the exact order requested by the client.

Legacy Interface (Do not change)

```
1 class LegacyInventorySystem {
2     public String fetchStock() {
3         return "ITEM:Rice|QTY:50|PRICE:70;ITEM:Oil|QTY:20|
4             PRICE:160;";
5     }
6 }
7 interface ModernApiResponse {
8     String getFormattedResponse();
9 }
```

Question 3: Restaurant Food Platter System

Duration: 35 minutes

A popular restaurant (famous for its Dim Sum and Fried Rice) allows customers to order food in two ways: individual items or predefined “Platters.”

A Platter consists of several individual food items. For example, a “Weekend Special Platter” might contain Fried Rice, two pieces of Dim Sum, and a slice of Red Velvet Cake. To encourage bulk purchases, any Platter automatically applies a 15% discount to the sum of its internal items.

Customers can place orders through different ordering channels:

- **Dine-in:** Adds a 10% service charge to the total bill.
- **Takeaway:** No additional charge.
- **Third-Party Delivery:** Adds a fixed 60 BDT delivery fee.

Your Task:

Model the menu items (individual foods and platters) such that the pricing logic treats them uniformly. Ensure that the ordering channel (Dine-in, Takeaway, Delivery) is decoupled from the food items so that new channels or food categories can be added independently without creating an explosion of subclasses.

Question 4: Smart Pet Care Device API

Duration: 25 minutes

A smart home system includes an automated Pet Feeder for a cat. The manufacturer provides an internal `HardwareFeeder` class with low-level methods like `dispenseFood(int grams)` and `playVoiceNote()`.

You are building a mobile application that needs a simpler, unified interface (`SmartPetController`) to interact with this hardware. Furthermore, to prevent accidental overfeeding, you must introduce a rate-limiting control mechanism: the feeder must not dispense food more than 4 times within a 24-hour window. If a 5th request is made, the system should block it and log a warning.

Additionally, the user can toggle a "Notification Mode" via the app, which sends an SMS to the user's phone whenever food is successfully dispensed.

Your Task:

Design this subsystem using the appropriate structural patterns. Ensure the rate-limiting logic controls access to the underlying hardware, while the notification system acts as a flexible add-on behavior.

Question 5: Football Statistics Aggregator

Duration: 30 minutes

A sports analytics app aggregates player statistics for the upcoming FIFA World Cup. The app sources data from multiple global providers.

- **Provider A** supplies data in XML format.
- **Provider B** supplies data as a serialized byte stream.

Your application strictly uses a `StandardPlayerStats` interface. You must integrate both providers.

Furthermore, users can view these stats in different UI modes: **Compact View** (shows only goals and assists) and **Detailed View** (shows pass accuracy, heat maps, and distance covered).

Your Task:

Use appropriate structural design patterns to bridge the gap between the disparate data providers and your standard interface, while keeping the data extraction logic completely independent of the UI display modes.

Question 6: University Course Management

Duration: 35 minutes

A university needs a system to manage its course structures. A full **Course** (e.g., Computer Architecture) is composed of several **Modules** (e.g., CPU Design, Memory Hierarchy). Each Module contains multiple individual **Topics** (e.g., Pipelining, Caching). Every Topic requires a specific number of credit hours to complete. The total credit hours for a Module is the sum of its Topics, and the total credit hours for a Course is the sum of its Modules.

To help students prepare, the university is introducing a **Review Session** feature. A review session can be wrapped around any individual Topic, an entire Module, or a full Course. Wrapping an entity in a Review Session adds exactly 2 extra credit hours to that specific entity.

Your Task:

Implement the system. A student should be able to query the total credit hours of their syllabus uniformly, whether they are looking at a single topic, a module, or the entire course, and regardless of whether review sessions have been added to specific parts of the hierarchy.

Question 7: Drone Fleet Control system

Duration: 35 minutes

A photography company uses a fleet of drones for aerial mapping. The company operates two different drone families:

- **DJI Air Series** (Proprietary flight controls)
- **Custom FPV Drones** (Open-source flight controls)

Regardless of the drone family, the drones are dispatched on specific **Route Types**:

- **Grid Scan**: Flies in a lawnmower pattern over a field.
- **Waypoint Navigation**: Flies point-to-point along rural roads.

To improve the captured images in real-time, the company can dynamically attach software **Filters** to the camera feed before saving the data:

- **Polarization Filter**: Removes glare.
- **High-Contrast Filter**: Enhances shadows and edge detection.

Your Task:

Design the drone control system. You must avoid a massive inheritance hierarchy (e.g., `DJIGridScanWithPolarization`). Ensure that the drone families and the route types can vary independently, and that image filters can be stacked (e.g., applying both Polarization and High-Contrast) dynamically at runtime.

Question 8: Inter-city Travel Booking

Duration: 30 minutes

A travel agency allows students to book trips between cities (e.g., Dhaka to Gopalganj). A trip booking is represented by a **BaseTrip** object containing the departure time and base fare.

Depending on the transport type and user preferences, the fare can be modified dynamically:

- **AC Coach:** Adds 300 BDT to the base fare.
- **Night Shift:** Applies a 10% surcharge to the current calculated fare.
- **Extra Luggage:** Adds a flat 150 BDT per additional bag.

Furthermore, to manage the backend processing of these trips, the system utilizes two different booking gateways: **Mobile Banking** (e.g., bKash) and **Bank Card**.

Your Task:

Implement a solution where trip features (AC, Night Shift, Luggage) can be combined arbitrarily and stacked on top of a **BaseTrip**. The finalized trip object must then be processed by the payment gateway, ensuring the gateway implementations are decoupled from the trip calculation logic. Output a final receipt showing the applied modifiers and the total fare.